

Natural Language Processing — Day 1

Text Preprocessing, Representation & Classical NLP

1. Text Preprocessing

Every NLP pipeline — no matter how advanced the model — starts with turning raw, messy text into a clean, consistent form. Garbage text in, garbage predictions out.

Tokenization

Splitting text into smaller units: words, sentences, or subwords.

```
import nltk
from nltk.tokenize import word_tokenize, sent_tokenize

text = "Deep Learning is powerful. NLP builds on it!"
print(word_tokenize(text))  # ['Deep', 'Learning', 'is', 'powerful', '.', 'NLP', ...]
print(sent_tokenize(text))  # ['Deep Learning is powerful.', 'NLP builds on it!']
```

Lowercasing & Removing Punctuation/Special Characters

```
import re
text = "Hello, World! NLP@2026 is Fun..."
clean = re.sub(r"[^a-zA-Z\s]", "", text.lower())
print(clean)  # hello world nlp is fun
```

Stopword Removal

Filtering out very common, low-information words ("the", "is", "and") that add little meaning to most NLP tasks.

```
from nltk.corpus import stopwords
stop_words = set(stopwords.words("english"))
tokens = ["this", "is", "an", "amazing", "nlp", "course"]
filtered = [w for w in tokens if w not in stop_words]
print(filtered)  # ['amazing', 'nlp', 'course']
```

Stemming vs Lemmatization

Both reduce words to a base form, but differently: **Stemming** crudely chops word endings using rules (fast, sometimes produces non-words); **Lemmatization** uses vocabulary and grammar rules to return the actual dictionary root.

```
from nltk.stem import PorterStemmer, WordNetLemmatizer

stemmer = PorterStemmer()
lemmatizer = WordNetLemmatizer()

print(stemmer.stem("studies"))  # "studi"    (chopped, not a real word)
print(lemmatizer.lemmatize("studies"))  # "study"    (real dictionary root)
```

Text Normalization

Additional cleanup: spelling correction, expanding contractions ("don't" → "do not"), and removing HTML tags or emojis before further processing.

```
import re
text = "I dont like this!! <br> so much noise 🚀"
text = text.replace("dont", "do not")
```

```
text = re.sub(r"<.*?>", "", text)          # remove HTML tags
text = re.sub(r"[^\w\s.,!?]", "", text)      # remove emojis/symbols
print(text)  # I do not like this!! so much noise
```

2. Text Representation (Feature Extraction)

Machine learning models need numbers, not raw text — these techniques convert cleaned text into numerical features.

Bag of Words (BoW)

Represents a document as word counts, completely ignoring word order.

```
from sklearn.feature_extraction.text import CountVectorizer

docs = ["I love NLP", "NLP loves me"]
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(docs)
print(vectorizer.get_feature_names_out()) # ['love' 'loves' 'me' 'nlp']
print(X.toarray()) # [[1 0 0 1], [0 1 1 1]]
```

N-grams

Captures short local context by grouping consecutive words — unigrams (1 word), bigrams (2 words), trigrams (3 words).

```
vectorizer = CountVectorizer(ngram_range=(1, 2)) # unigrams + bigrams
X = vectorizer.fit_transform(["I love NLP"])
print(vectorizer.get_feature_names_out())
# ['love' 'love nlp' 'nlp']
```

TF-IDF (Term Frequency – Inverse Document Frequency)

Weighs each word by how frequent it is in a document (TF) versus how rare it is across all documents (IDF) — common words across all documents get down-weighted, distinctive words get boosted.

```
from sklearn.feature_extraction.text import TfidfVectorizer

docs = ["the cat sat", "the dog ran fast"]
tfidf = TfidfVectorizer()
X = tfidf.fit_transform(docs)
print(tfidf.get_feature_names_out())
print(X.toarray().round(2))
```

One-Hot Encoding for Text

Represents each word as a vector with a single 1 and all other positions 0 — simple, but produces extremely sparse, high-dimensional vectors with no notion of similarity.

Limitations of BoW / N-grams / TF-IDF / One-Hot

- Sparse, high-dimensional vectors (one dimension per vocabulary word)
- No semantic meaning — "good" and "great" are just as "different" as "good" and "car"
- Vocabulary size grows without bound as more documents are added

This is exactly the gap that Word Embeddings were built to close.

3. Word Embeddings

Embeddings represent words as **dense** vectors (typically 100-300 dimensions) where semantically similar words end up close together in vector space — unlike BoW/TF-IDF, which are sparse and meaning-blind.

Word2Vec — CBOW vs Skip-gram

Variant	Predicts	Best for
CBOW (Continuous Bag of Words)	Target word from surrounding context words	Faster, works well with frequent words
Skip-gram	Surrounding context words from the target word	Works better with smaller datasets / rare words

```
from gensim.models import Word2Vec

sentences = [["deep", "learning", "is", "powerful"], ["nlp", "uses", "deep", "learning"]]
model = Word2Vec(sentences, vector_size=50, window=3, sg=1, min_count=1) # sg=1 -> skip-gram
print(model.wv["learning"]) # 50-dim dense vector
print(model.wv.most_similar("deep")) # words closest in meaning
```

GloVe (Global Vectors)

Learns embeddings from **global word co-occurrence statistics** across the entire corpus, rather than only local context windows like Word2Vec — captures broader corpus-level patterns.

FastText

Extends Word2Vec by learning embeddings for **subword units (character n-grams)** rather than whole words — this means it can generate a reasonable embedding for a word it has never seen before (out-of-vocabulary / OOV words), which Word2Vec and GloVe cannot do.

Cosine Similarity

Measures how "close" two word vectors are by the angle between them (not their magnitude) — the standard way to quantify semantic similarity between embeddings.

```
from numpy import dot
from numpy.linalg import norm

def cosine_similarity(v1, v2):
    return dot(v1, v2) / (norm(v1) * norm(v2))

# similarity closer to 1 = more similar meaning
```

Static vs Contextual Embeddings

Word2Vec/GloVe/FastText produce **static** embeddings — one fixed vector per word, regardless of context. This fails for words with multiple meanings: "bank" (river) and "bank" (money) get the identical vector. **Contextual embeddings** (produced by models like BERT) generate a different vector for the same word depending on its surrounding sentence — this is covered in Day 2.

4. Classical NLP Tasks

Text Classification

Assigning a category to a piece of text — e.g. spam detection, sentiment analysis — typically using BoW/TF-IDF features with Naive Bayes, Logistic Regression, or SVM.

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB

X = TfidfVectorizer().fit_transform(["free money now", "meeting at noon"])
y = [1, 0] # 1 = spam, 0 = not spam
clf = MultinomialNB().fit(X, y)
```

POS Tagging (Part-of-Speech)

Labeling each word with its grammatical role — noun, verb, adjective, etc.

```
import spacy
nlp = spacy.load("en_core_web_sm")
doc = nlp("Deep learning models process text")
for token in doc:
    print(token.text, "->", token.pos_)
# Deep -> ADJ, learning -> NOUN, models -> NOUN, process -> VERB, text -> NOUN
```

Named Entity Recognition (NER)

Identifying and classifying real-world entities in text — people, organizations, locations, dates, etc.

```
doc = nlp("Sara works at Google in London")
for ent in doc.ents:
    print(ent.text, "->", ent.label_)
# Sara -> PERSON, Google -> ORG, London -> GPE
```

Parsing — Dependency vs Constituency

Type	What it captures
Dependency Parsing	Grammatical relationships between words (subject, object, modifier)
Constituency Parsing	Hierarchical phrase structure (noun phrases, verb phrases, etc.)

Topic Modeling — LDA & NMF

Unsupervised techniques that discover hidden themes/topics across a collection of documents, without any labels.

```
from sklearn.decomposition import LatentDirichletAllocation
from sklearn.feature_extraction.text import CountVectorizer

X = CountVectorizer().fit_transform(documents)
lda = LatentDirichletAllocation(n_components=5, random_state=42)
lda.fit(X) # discovers 5 latent topics across the document collection
```

N-gram Language Models & Perplexity

An N-gram language model estimates the probability of a word given the previous N-1 words. **Perplexity** measures how "surprised" the model is by real text — lower perplexity means the model predicts the actual next words more confidently.

Perplexity = $2^{-(1/N) * \sum(\log_2(P(\text{word}_i | \text{context})))}$
Lower perplexity -> better language model fit on that text

Key Takeaways

- Every NLP pipeline starts with **preprocessing** — tokenization, cleaning, stopword removal, stemming/lemmatization, normalization.
- **BoW, N-grams, and TF-IDF** turn text into numbers but stay sparse and blind to meaning.
- **Word Embeddings** (Word2Vec, GloVe, FastText) solve the meaning problem with dense vectors where similar words sit close together.
- The jump from TF-IDF to Word2Vec is the jump from **counting** words to **understanding** them.
- **Static embeddings** assign one fixed vector per word; **contextual embeddings** (Day 2: BERT) adapt the vector to the surrounding sentence.
- **Classical NLP tasks** — classification, POS tagging, NER, parsing, topic modeling, N-gram language models — remain the foundation that modern Transformer-based systems build on top of.

Next (Day 2): Transformers, BERT, GPT, applied NLP tasks (translation, summarization, QA), evaluation metrics, and the bridge into LLMs & RAG.